ODOOSKILLS • GUIDE GRATUIT

Édition Odoo 20

Premier *module* Odoo 20

Odoo 20 pour développeurs : votre premier module, pas à pas

> fil rouge : Labo Horizon, code public sur GitHub



BENHAMIDA MUSTAPHA

ODOOSKILLS.COM · ÉDITION 2026-2027

CODE SOURCE PUBLIC · COMMUNITY

GUIDE GRATUIT · DÉVELOPPEMENT · 20.0

Guide gratuit

Odoo 20 pour développeurs : votre premier module — Mustapha Benhamida, OdooSkills. Édition 2026-2027, Odoo 20.

© 2026 Mustapha Benhamida. Ce guide peut être partagé librement, à l'identique et sans modification.

Le code de Labo Horizon est publié sous licence LGPL-3 : `github.com/stagenie/labo-horizon` (tag `ch02`). Labo Horizon est un laboratoire fictif : patients et résultats sont inventés.

Odoo est une marque déposée d'Odoo S.A. Ce guide est indépendant : il n'est ni édité, ni approuvé, ni affilié à Odoo S.A.

OdooSkills · `https://odooskills.com`

Sommaire

Chapitre 0 — Environnement Odoo 20 1

0.1 — Les quatre briques 1

0.2 — Installer les briques 2

0.3 — Le fichier `odoo.conf` 3

0.4 — Première base 4

Chapitre 1 — Le premier module 6

1.1 — L'arborescence 6

1.2 — Le manifeste 7

1.3 — Le modèle `lab.patient` 8

1.4 — Installer et mettre à jour 9

Chapitre 2 — Les champs 11

2.1 — La fiche patient complète 11

2.2 — Le catalogue des analyses 12

Chapitre 3 — Vues, action et menu 15

3.1 — Liste et formulaire 15

3.2 — Action et menu 17

Chapitre 4 — Les droits d'accès en Odoo 20 21

4.1 — Un seul modèle : `ir.access` 21

4.2 — Le piège de la ligne sans groupe 22

Chapitre 5 — Les dix pannes du passage de 19 à 20 24

5.1 — Les pannes silencieuses 24

5.2 — Les pannes bruyantes 25

5.3 — La méthode de portage 26

Chapitre 6 — Checklist « mon premier module » 27

Les commandes de référence 28

Chapitre 7 — Et ensuite 29

7.1 — Le code 29

7.2 — Les articles du blog OdooSkills 29

7.3 — L'ebook *Développer sur Odoo 20* 30

Chapitre 0 — Environnement Odoo 20

Ce guide mène de zéro à un premier module Odoo 20 installé, avec ses écrans, ses droits et ses données de démonstration. Il s'adresse au développeur Python qui découvre Odoo comme au développeur Odoo 17, 18 ou 19 qui passe à la version 20. Le fil rouge est *Labo Horizon*, un laboratoire d'analyses médicales fictif : le module `lab_core` enregistre ses patients et son catalogue d'analyses.

Tout le code imprimé dans ce guide est publié sous licence LGPL-3 dans le dépôt public `github.com/stagenie/labo-horizon`, au tag `ch02`. Il a été installé sur une base Odoo 20 et ses quinze tests passent.

0.1 — Les quatre briques

Un environnement Odoo 20 de développement assemble quatre briques :

- **PostgreSQL** conserve toutes les données, une base par instance ;
- **Python** exécute le serveur Odoo et le code des modules ;

- **le serveur Odoo** (`odoo-bin`) sert l'interface web, applique les règles métier et les droits d'accès ;
- **le navigateur** affiche l'interface ; aucun logiciel client n'est à installer.

Odoo 20 relève le niveau minimal des deux premières. Le fichier `odoo/release.py` du code source le fixe :

```
MIN_PY_VERSION = (3, 12)
MIN_PG_VERSION = 16
```

Ubuntu 24.04 LTS fournit exactement ces versions dans ses dépôts, Python 3.12 et PostgreSQL 16 : c'est la distribution retenue ici. Sous Windows, Ubuntu 24.04 s'installe dans WSL 2 et la procédure reste identique.

0.2 — Installer les briques

```
sudo apt update
sudo apt install -y postgresql postgresql-contrib \
    python3.12 python3.12-dev python3.12-venv build-essential \
    libpq-dev libxml2-dev libxslt1-dev libldap2-dev libsasl2-dev \
    libjpeg-dev zlib1g-dev git
sudo -u postgres createuser -s $USER
```

La dernière ligne crée un rôle PostgreSQL du même nom que l'utilisateur Linux : Odoo s'y connecte alors par la *socket* locale, sans mot de passe. L'option `-s` en fait un superutilisateur, confort d'un poste de développement ; un serveur de production utilise un rôle dédié, sans ce privilège.

Le code d'Odoo 20 *Community* se clone depuis GitHub, sans son historique, puis ses dépendances s'installent dans un environnement virtuel :

```
mkdir -p ~/odoo20 && cd ~/odoo20
git clone --depth 1 -b 20.0 https://github.com/odoo/odoo.git
python3.12 -m venv venv
source venv/bin/activate
pip install --upgrade pip wheel
pip install -r odoo/requirements.txt
git clone https://github.com/stagenie/labo-horizon.git
git -C labo-horizon checkout ch02
```

Le code d'Odoo (`odoo/`) et celui du laboratoire (`labo-horizon/`) vivent dans deux dossiers distincts : aucun module n'est jamais écrit dans `odoo/`, qu'une mise à jour effacerait.

0.3 — Le fichier `odoo.conf`

```
[options]
addons_path = ~/odoo20/odoo/addons,
    ~/odoo20/labo-horizon
http_port = 8069
dev_mode = reload,xml
workers = 0
```

- **`addons_path`** liste les dossiers où Odoo cherche des modules, séparés par des virgules.
- **Aucune ligne `db_*`** : sans hôte ni utilisateur, Odoo passe par la *socket* locale avec le rôle créé plus haut.
- **`dev_mode = reload,xml`** : `reload` redémarre le serveur quand un fichier Python change ; `xml` lit les vues dans les fichiers du module, sans mise à jour à chaque retouche.
- **`workers = 0`** : un seul processus, celui du débogueur pas à pas.

> **Attention**
>
> La ligne `http_interface` est absente, et c'est voulu sur un poste de développement : sa valeur par défaut en Odoo 20 est `127.0.0.1` (`odoo/tools/config.py:289`). Le serveur ne répond qu'à la machine elle-même. Sur un serveur distant joint par son adresse IP, la page reste injoignable sans la moindre erreur dans le journal : il faut alors écrire `http_interface = 0.0.0.0`, ou placer un *reverse proxy* sur la même machine.

0.4 — Première base

```
cd ~/odoo20
./odoo/odoo-bin -c odoo.conf -d vs20_lab_dev -i base \
    --with-demo --load-language=fr_FR --stop-after-init
./odoo/odoo-bin -c odoo.conf -d vs20_lab_dev
```

La première commande crée la base `vs20_lab_dev`, installe le module `base`, charge la démonstration et le français, puis rend la main. L'option `--with-demo` est indispensable : sans elle, une base neuve n'a aucune donnée de démonstration. La seconde démarre le serveur ; l'adresse `http://localhost:8069` ouvre l'écran de connexion, identifiant `admin`, mot de passe `admin`.

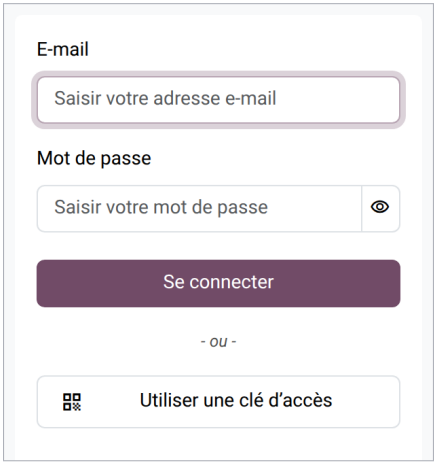


Figure 0.1 — L'écran de connexion de la base `vs20_lab_dev`.

Chapitre 1 — Le premier module

Un module Odoo est un dossier Python placé dans l'un des chemins d'`addons_path`. Il contient au minimum deux fichiers : `__init__.py`, qui en fait un paquet Python, et `__manifest__.py`, qui le décrit à Odoo. Tout le reste (modèles, vues, droits, données) est déclaré depuis ces deux points d'entrée.

1.1 — L'arborescence

```
lab_core/
├── __init__.py               # importe le paquet models
├── __manifest__.py           # carte d'identité du module
├── data/                     # catalogue d'analyses
├── demo/                     # patients fictifs
├── models/
│   ├── __init__.py
│   ├── lab_patient.py        # le modèle lab.patient
│   └── lab_analysis.py       # le modèle lab.analysis
├── security/
│   └── ir.access.csv         # droits d'accès (Odoo 20)
├── tests/
└── views/                    # listes, formulaires, actions, menus
```

Les fichiers ne se chargent pas d'eux-mêmes. Le code Python est trouvé par les `import` en cascade depuis `__init__.py` ; les fichiers de données (XML, CSV) ne le sont que s'ils figurent dans le manifeste.

lab_core/__init__.py

```
from . import models
```

lab_core/models/__init__.py

```
from . import lab_patient
from . import lab_analysis
```

> **Astuce**
> Un fichier Python oublié dans un `__init__.py` ne provoque aucune erreur : son code n'est jamais exécuté et le modèle qu'il définit n'existe pas. Le premier symptôme apparaît plus loin, au chargement d'une vue ou d'un fichier de droits qui le cite.

1.2 — Le manifeste

Le manifeste est un dictionnaire Python littéral : Odoo le lit sans exécuter le module.

lab_core/__manifest__.py

```
{
    'name': 'Labo Horizon — Laboratoire',
    'version': '20.0.1.2.0',
    'summary': "Patients, demandes d'analyses, résultats et comptes rendus",
    'category': 'Services',
    'author': 'OdooSkills',
    'website': 'https://odooskills.com',
    'license': 'LGPL-3',
    'depends': ['base'],
    'data': [
```

```
        'security/ir.access.csv',
        'data/lab_analysis_data.xml',
        'views/lab_patient_views.xml',
        'views/lab_catalog_views.xml',
        'views/lab_menus.xml',
    ],
    'demo': ['demo/lab_demo.xml'],
    'application': True,
}
```

- **`version`** commence par la série d'Odoo, `20.0`. Odoo 20 refuse d'installer un module d'une autre série : il le déclare non installable, avec l'avertissement `The module lab_core has an incompatible version, setting installable=False`. Un module 19 copié tel quel est donc invisible, sans erreur.
- **`depends`** liste les modules à installer avant celui-ci. `base` est présent dans toute base.
- **`data`** liste les fichiers chargés à chaque installation et mise à jour, **dans l'ordre** : les droits d'abord, les vues ensuite, les menus en dernier, car un menu cite une action qui doit déjà exister.
- **`demo`** liste les fichiers chargés seulement dans une base créée avec `--with-demo`.
- **`application`** à `True` présente le module comme une application dans l'écran *Apps*.

1.3 — Le modèle `lab.patient`

Un **modèle** est une classe Python qui hérite de `models.Model`. Son attribut `_name` en fixe le nom technique ; à l'installation, l'ORM crée la table PostgreSQL correspondante (`lab_patient`) et une colonne par champ stocké.

lab_core/models/lab_patient.py

```
from odoo import fields, models


class LabPatient(models.Model):
    _name = 'lab.patient'
    _description = 'Patient'
    _order = 'name'

    name = fields.Char('Nom', required=True)
    birthdate = fields.Date('Date de naissance')
    phone = fields.Char('Téléphone')
    # ...
```

- **`_name`** : le nom technique, celui qu'utilisent le code (`self.env['lab.patient']`), les vues et les droits.
- **`_description`** : le libellé lisible du modèle, repris dans les messages d'erreur.
- **`_order`** : le tri par défaut des recherches et des listes ; sans lui, Odoo trie par ordre de création.
- Le premier argument de chaque champ est son libellé à l'écran. `required=True` rend le nom obligatoire, dans le formulaire comme en base (contrainte `NOT NULL`).

La table compte en réalité plus de colonnes que de champs déclarés : l'ORM ajoute d'office `id`, ainsi que `create_uid`, `create_date`, `write_uid` et `write_date`, qui tracent qui a créé et modifié chaque enregistrement, et quand.

1.4 — Installer et mettre à jour

```
./odoo/odoo-bin -c odoo.conf -d vs20_lab_dev -i lab_core --stop-after-init
./odoo/odoo-bin -c odoo.conf -d vs20_lab_dev -u lab_core --stop-after-init
```

`-i` installe le module, `-u` le met à jour. Une mise à jour est nécessaire après toute modification d'un modèle Python (nouveau champ) ou d'un fichier de données ; avec `dev_mode = xml`, une retouche de vue se voit sans elle, au simple rechargement de la page.

Une fois le module installé, l'application *Laboratoire* apparaît dans le menu des applications, et son menu *Patients* ouvre la liste des dix patients de démonstration.

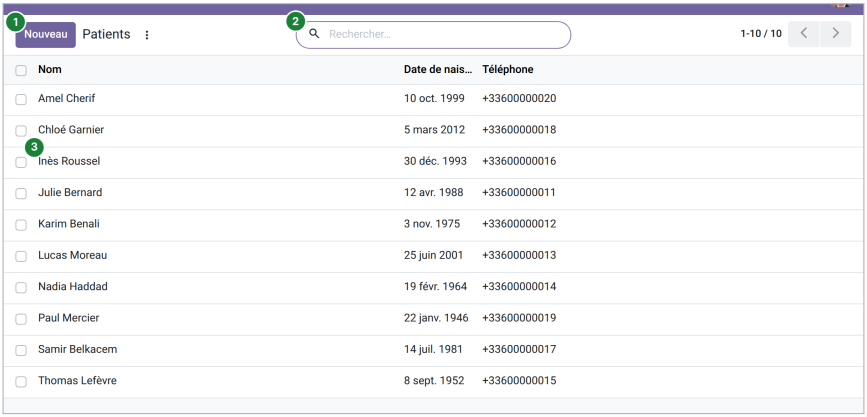


Figure 1.1 — La liste des patients, telle qu'elle se présente avec les trois premiers champs : le bouton Nouveau (1), la barre de recherche (2) et une ligne, qui ouvre le formulaire (3).

Chapitre 2 — Les champs

Un champ est un attribut de classe, objet du module `fields`. Son type décide de la colonne PostgreSQL, du composant affiché à l'écran et des valeurs acceptées. Ce chapitre complète la fiche patient et crée un second modèle, le catalogue des analyses du laboratoire.

2.1 — La fiche patient complète

lab_core/models/lab_patient.py

```
    name = fields.Char('Nom', required=True)
    birthdate = fields.Date('Date de naissance')
    phone = fields.Char('Téléphone')
    gender = fields.Selection([('female', 'Femme'), ('male', 'Homme')], 'Sexe')
    email = fields.Char('E-mail')
    notes = fields.Html('Notes internes', help="Informations utiles à l'accueil
↪ ; jamais transmises au patient.")
```

Type	Contenu	Composant à l'écran
`Char`	texte court	champ de saisie d'une ligne

Type	Contenu	Composant à l'écran
`Date`	date sans heure	sélecteur de date
`Selection`	une valeur parmi une liste de couples (clé, libellé)	liste déroulante, ou boutons radio
`Html`	texte mis en forme	éditeur de texte riche
`Float`	nombre décimal	champ numérique
`Boolean`	vrai ou faux	case à cocher
`Binary`	fichier	bouton de téléversement

Tableau 2.1 — Les types de champs simples utilisés par `lab_core`.

Pour un `Selection`, seule la clé (`female`, `male`) est stockée en base ; le libellé est traduisible et peut changer sans migration. L'attribut `help` affiche une bulle d'aide au survol du libellé.



Figure 2.1 — La fiche de Julie Bernard : date de naissance, sexe en boutons radio, téléphone et e-mail à droite, notes internes dans l'onglet Notes.

2.2 — Le catalogue des analyses

Le second modèle décrit les analyses que le laboratoire sait réaliser : un code, un libellé, un type de prélèvement, une unité et des valeurs de référence.

lab_core/models/lab_analysis.py

```
from odoo import fields, models

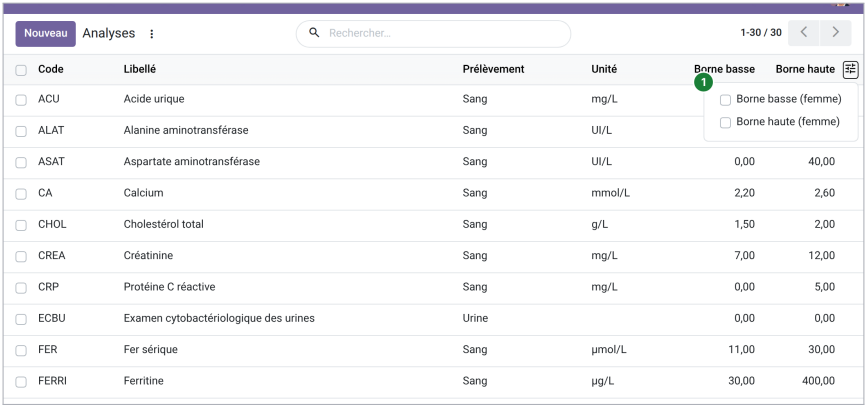
SAMPLE_TYPES = [('blood', 'Sang'), ('urine', 'Urine'), ('swab', 'Écouvillon')]

class LabAnalysis(models.Model):
    _name = 'lab.analysis'
    _description = 'Analyse'
    _order = 'code'
    _rec_names_search = ('code', 'name')

    code = fields.Char('Code', required=True)
    name = fields.Char('Libellé', required=True, translate=True)
    sample_type = fields.Selection(SAMPLE_TYPES, 'Prélèvement', required=True,
↪ default='blood')
    unit = fields.Char('Unité')
    ref_min = fields.Float('Borne basse', digits=(10, 2))
    ref_max = fields.Float('Borne haute', digits=(10, 2))
    # ...
    protocol = fields.Binary('Fiche technique', attachment=True)
    protocol_filename = fields.Char('Nom du fichier')
    active = fields.Boolean(default=True)
```

- **`_rec_names_search`** : les champs interrogés quand un utilisateur tape dans un champ de recherche ou de sélection. Ici, « CREA » comme « Créatinine » retrouvent la même analyse.
- **`translate=True`** : le libellé se traduit par langue, sans changer le code de l'analyse.
- **`default='blood'`** : la valeur proposée à la création.
- **`digits=(10, 2)`** : dix chiffres au total, dont deux après la virgule.
- **`attachment=True`** : le fichier est stocké comme pièce jointe, hors de la table. En Odoo 20, un champ `Binary` se lit et s'écrit en octets bruts, plus en base64.
- **`active`** : un champ booléen de ce nom active l'archivage. Une analyse archivée disparaît des listes et des recherches sans être supprimée.

Le catalogue est chargé par un fichier de données, `data/lab_analysis_data.xml`, listé sous `data` dans le manifeste : il est donc présent dans toute base, démonstration ou non.



Code	Libellé	Prélèvement	Unité	Borne basse	Borne haute
ACU	Acide urique	Sang	mg/L		
ALAT	Alanine aminotransférase	Sang	UI/L		
ASAT	Aspartate aminotransférase	Sang	UI/L	0,00	40,00
CA	Calcium	Sang	mmol/L	2,20	2,60
CHOL	Cholestérol total	Sang	g/L	1,50	2,00
CREA	Créatinine	Sang	mg/L	7,00	12,00
CRP	Protéine C réactive	Sang	mg/L	0,00	5,00
ECBU	Examen cytobactériologique des urines	Urine		0,00	0,00
FER	Fer sérique	Sang	µmol/L	11,00	30,00
FERRI	Ferritine	Sang	µg/L	30,00	400,00

Figure 2.2 — Les 30 analyses du catalogue, triées par code. Le bouton de colonnes (1) affiche les bornes propres aux femmes, masquées par défaut.

Chapitre 3 — Vues, action et menu

Une vue décrit comment présenter un modèle à l'écran. Elle s'enregistre en base comme n'importe quelle donnée : un enregistrement du modèle `ir.ui.view`, déclaré dans un fichier XML du module.

3.1 — Liste et formulaire

lab_core/views/lab_patient_views.xml

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
    <record id="lab_patient_view_list" model="ir.ui.view">
        <field name="name">lab.patient.list</field>
        <field name="model">lab.patient</field>
        <field name="arch" type="xml">
            <list>
                <field name="name"/>
                <field name="birthdate"/>
                <field name="phone"/>
                <field name="gender"/>
                <field name="email"/>
            </list>
        </field>
    </record>
```

Le formulaire suit, dans le même fichier :

lab_core/views/lab_patient_views.xml

```
    <record id="lab_patient_view_form" model="ir.ui.view">
        <field name="name">lab.patient.form</field>
        <field name="model">lab.patient</field>
        <field name="arch" type="xml">
            <form>
                <sheet>
                    <div class="oe_title">
                        <h1><field name="name" placeholder="Nom du
↪ patient"/></h1>
                    </div>
                    <group>
                        <group>
                            <field name="birthdate"/>
                            <field name="gender" widget="radio"/>
                        </group>
                        <group>
                            <field name="phone" widget="phone"/>
                            <field name="email" widget="email"/>
                        </group>
                    </group>
```

```
                <notebook>
                    <page name="notes" string="Notes">
                        <field name="notes"/>
                    </page>
                </notebook>
            </sheet>
        </form>
    </field>
</record>
```

- **`<list>`** affiche une colonne par champ cité, dans l'ordre d'écriture. La balise s'appelait `<tree>` jusqu'à Odoo 17.
- **`<form>`** présente un enregistrement à la fois ; `<sheet>` dessine la feuille blanche, le bloc `oe_title` met le nom en titre.
- **Deux `<group>` dans un `<group>`** produisent deux colonnes de champs, celles de la figure 2.1.
- **`<notebook>`** et **`<page>`** créent des onglets.
- **`widget`** change la présentation d'un champ : `radio` pour la sélection, `phone` et `email` pour des liens cliquables.

Les libellés ne sont pas répétés dans la vue : ils viennent du premier argument de chaque champ Python.

3.2 — Action et menu

Une vue ne s'affiche pas seule. Une **action de fenêtre** dit quel modèle ouvrir et sous quelles formes ; un **menu** déclenche l'action.

lab_core/views/lab_patient_views.xml

```
<record id="lab_patient_action" model="ir.actions.act_window">
    <field name="name">Patients</field>
    <field name="res_model">lab.patient</field>
    <field name="view_mode">list,form</field>
</record>
```

lab_core/views/lab_menus.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<odoo>
    <menuitem id="menu_lab_root" name="Laboratoire" sequence="40"
              web_icon="lab_core,static/description/icon.png"/>
    <menuitem id="menu_lab_patient" name="Patients" parent="menu_lab_root"
              action="lab_patient_action" sequence="10"/>
    <menuitem id="menu_lab_config" name="Configuration" parent="menu_lab_root"
 ↪ sequence="90"/>
    <menuitem id="menu_lab_analysis" name="Analyses" parent="menu_lab_config"
              action="lab_analysis_action" sequence="10"/>
</odoo>
```

`view_mode` liste les types de vues proposés, le premier étant celui d'ouverture. L'action ne cite pas ses vues : Odoo retient, pour chaque type, la vue du modèle de plus petite priorité. Le menu sans parent devient une **application** du menu principal ; ses enfants forment la barre de l'application.

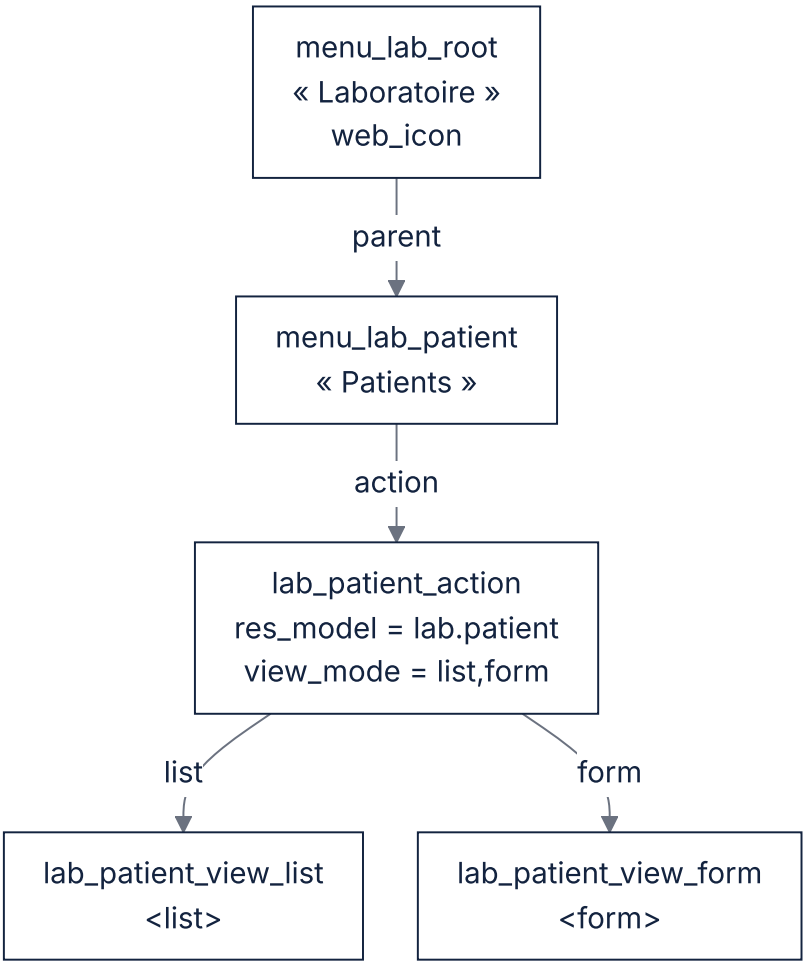


Figure 3.1 — Le menu « Patients » déclenche l'action, qui ouvre `lab.patient` en liste puis en formulaire.

Astuce

Un menu n'est affiché que si l'utilisateur peut lire le modèle de son action. Sans droit de lecture sur `lab.patient`, toute l'application *Laboratoire* disparaît du menu, sans message : le premier réflexe devant un menu invisible est de relire le fichier de droits.

Chapitre 4 — Les droits d'accès en Odoo 20

Un modèle sans droits d'accès existe en base, mais aucun utilisateur ordinaire ne peut le lire. Les droits font donc partie du module, au même titre que ses champs. C'est aussi le changement d'Odoo 20 qui bloque le plus souvent l'installation d'un module venu d'Odoo 19.

4.1 — Un seul modèle : `ir.access`

Jusqu'à Odoo 19, deux modèles portaient la sécurité : `ir.model.access` (les droits par modèle, fichier `ir.model.access.csv`) et `ir.rule` (les règles par enregistrement). En Odoo 20, ils n'existent plus. Un seul modèle, `ir.access` (`odoo/addons/base/models/ir_access.py:66`), porte tout : chaque ligne associe un **modèle**, un **groupe**, des **opérations** et, au besoin, un **domaine** qui restreint les enregistrements concernés.

```
lab_core/security/ir.access.csv
```

```
id,name,model_id,group_id/id,operation,domain
access_lab_patient_user,Patient : utilisateur interne,lab.patient,base.group_
↪ user,crud,
access_lab_analysis_user,Analyse : utilisateur
↪ interne,lab.analysis,base.group_user,crud,
```

- **`id`** : l'identifiant externe de la ligne, unique dans le module.
- **`model_id`** : le nom technique du modèle, tel quel.
- **`group_id/id`** : le groupe concerné ; `base.group_user` désigne les utilisateurs internes.
- **`operation`** : les lettres des opérations permises, dans l'ordre `crud` : créer, lire (*read*), modifier (*update*), supprimer (*delete*). `r` donne la lecture seule, `cru` tout sauf la suppression.
- **`domain`** : vide ici. Renseigné, il limite la ligne à certains enregistrements, ce que faisait une `ir.rule`.

Un module Odoo 19 qui déclare encore `security/ir.model.access.csv` échoue à l'installation sur `KeyError: 'ir.model.access'`.

4.2 — Le piège de la ligne sans groupe

> **Attention**
>
> Une ligne dont la colonne `group_id/id` est vide ne donne pas l'accès à tous : en Odoo 20, elle devient une **restriction** qui s'applique à tous. L'utilisateur interne lui-même reçoit alors l'erreur `You are not allowed to access 'Patient' (lab.patient) records.` Pour ouvrir un modèle à chacun, citer explicitement le groupe `base.group_everyone`.

Un fichier de droits oublié, ou réduit à son en-tête, a un effet voisin : le modèle s'installe, mais personne en dehors de l'administrateur technique ne peut l'ouvrir, et Odoo 20 n'émet plus d'avertissement au chargement. Le fichier de droits se place donc **en tête** de la liste `data` du manifeste, et se teste avec un utilisateur ordinaire, jamais seulement avec l'administrateur.

Chapitre 5 — Les dix pannes du passage de 19 à 20

Un module écrit pour Odoo 19 ne s'installe pas tel quel sur Odoo 20 : sa version `19.0.x` le rend non installable (chapitre 1). Une fois la version corrigée, dix changements expliquent l'essentiel des pannes. Les cinq premiers sont **silencieux** : aucune erreur, mais un résultat faux. Les cinq suivants sont **bruyants** : l'installation ou l'exécution s'arrête sur une erreur explicite.

5.1 — Les pannes silencieuses

1. `t-esc` et `t-raw` ne rendent plus rien. Dans un gabarit QWeb côté serveur (rapport PDF, e-mail, page web), ces deux directives produisent une balise vide ; le journal n'émet qu'un avertissement `Unknown directives or unused attributes` (`odoo/addons/base/models/ir_qweb.py:1929`).

```
<span t-esc="doc.name"/>   <!-- Odoo 19 -->
<span t-out="doc.name"/>   <!-- Odoo 20 -->
```

2. `t-set` dans le corps d'un `t-call` ne transmet plus la variable. Le sous-gabarit la reçoit vide, sans aucun avertissement. La variable se passe en attribut du `t-call`.

```
<t t-call="module.bloc"><t t-set="titre" t-value="doc.name"/></t>   <!-- Odoo
↪ 19 -->
<t t-call="module.bloc" titre="doc.name"/>                           <!-- Odoo
↪ 20 -->
```

3. Une pièce jointe créée avec `datas=` est vide. Le champ `datas` (base64) est retiré : la pièce jointe est créée avec zéro octet et un simple `UserWarning: Use raw, datas has beeen removed` (`ir_attachment.py:535`). Le contenu se passe en octets bruts.

```
env['ir.attachment'].create({'name': 'cr.pdf', 'datas': base64.b64encode(pdf)})
↪ # Odoo 19
env['ir.attachment'].create({'name': 'cr.pdf', 'raw': pdf})
↪ # Odoo 20
```

4. Une ligne de droits sans groupe restreint tout le monde. Détaillé au chapitre 4 : le groupe `base.group_everyone` remplace la colonne vide.

5. Le serveur n'écoute que sur `127.0.0.1`. Valeur par défaut de `http_interface` en Odoo 20 (`odoo/tools/config.py:289`). Un serveur joint par son adresse IP reste muet, sans erreur au journal. Remède : `http_interface = 0.0.0.0`, ou un *reverse proxy* local.

5.2 — Les pannes bruyantes

6. `ir.model.access` et `ir.rule` n'existent plus. Un fichier `ir.model.access.csv` arrête l'installation sur `KeyError: 'ir.model.access'`. Remède : le fichier `security/ir.access.csv` du chapitre 4.

7. Le paquet `odoo.osv` est supprimé. Tout import `from odoo.osv import expression` lève `ModuleNotFoundError`. Les domaines se manipulent avec la classe `Domain`.

```
from odoo.osv import expression   # Odoo 19
from odoo.fields import Domain    # Odoo 20
```

8. `env.registry.clear_cache()` est supprimé. L'appel lève `AttributeError`. Son remplaçant est `env.transaction.invalidate_ormcache()` (`odoo/orm/environments.py:833`).

9. `pytz` n'est plus une dépendance. Le module ne figure plus dans `requirements.txt`. Les fuseaux horaires passent par `zoneinfo`, de la bibliothèque standard ; `env.tz` est un objet `ZoneInfo`.

10. OWL 3 côté navigateur. Le framework JavaScript passe en version 3 : `this.` devient obligatoire dans les gabarits des composants (`this.props.x`, `t-on-click="this.onClick"`), les fonctions `useState` et `useRef` disparaissent, et jQuery comme FontAwesome ne sont plus fournis. Un composant Odoo 19 se réécrit plus qu'il ne se corrige.

5.3 — La méthode de portage

Un module à la fois, dans l'ordre de ses dépendances : passer la version à `20.0.1.0.0`, convertir d'abord les droits (c'est ce qui bloque l'installation), installer sur une base de test, **lire le journal et regarder l'écran**, puis lancer les tests du module. Les pannes silencieuses ne se voient qu'à l'écran ou dans les tests : un journal propre ne prouve rien à lui seul.

Chapitre 6 — Checklist « mon premier module »

Les douze étapes ci-dessous résument le guide. Chacune se coche avant la suivante.

1. Python 3.12 et PostgreSQL 16 installés ; `python3.12 --version` et `SELECT version();` le confirment.
2. Code d'Odoo 20 cloné (branche `20.0`), environnement virtuel actif, `requirements.txt` installé.
3. `odoo.conf` écrit, avec le dossier des modules personnels dans `addons_path`.
4. Base de développement créée avec `--with-demo`.
5. Dossier du module créé, avec `__init__.py` et `__manifest__.py`.
6. Version du manifeste en `20.0.x.y.z`, licence déclarée, `depends` renseigné.
7. Modèle déclaré, avec `_name` et `_description`, et importé depuis `models/__init__.py`.
8. `security/ir.access.csv` écrit, chaque ligne avec un groupe, et placé en tête de `data`.
9. Vues liste et formulaire, action, puis menus, dans cet ordre dans `data`.

10. Module installé avec `-i`, journal relu : aucune ligne `ERROR` ni `WARNING` liée au module.
11. Écrans vérifiés avec un utilisateur interne ordinaire, pas seulement avec l'administrateur.
12. Tests lancés et verts.

Les commandes de référence

```
# installer, puis mettre à jour un module
./odoo/odoo-bin -c odoo.conf -d vs20_lab_dev -i lab_core --stop-after-init
./odoo/odoo-bin -c odoo.conf -d vs20_lab_dev -u lab_core --stop-after-init

# lancer les tests du module
./odoo/odoo-bin -c odoo.conf -d vs20_lab_test -i lab_core \
    --test-enable --test-tags=/lab_core --stop-after-init

# démarrer le serveur
./odoo/odoo-bin -c odoo.conf -d vs20_lab_dev
```

En fin de run de tests, la ligne `odoo.tests.result: 0 failed, 0 error(s)` confirme le succès ; une ligne `odoo.tests.stats: lab_core: 15 tests` donne le nombre de tests du module.

Chapitre 7 — Et ensuite

Le module `lab_core` de ce guide est la première pierre d'un système complet : demandes d'analyses, résultats, calculs et contrôles, chatter et activités, rapports PDF, portail patient. Trois chemins mènent plus loin.

7.1 — Le code

Le dépôt public `github.com/stagenie/labo-horizon` contient un tag par étape : `ch02` correspond à ce guide, les tags suivants ajoutent les relations entre modèles, les champs calculés, les contraintes et le cycle de vie des demandes. Chaque tag s'installe sur une base Odoo 20 et passe ses tests.

7.2 — Les articles du blog OdooSkills

- **Installer Odoo 20 sur Ubuntu avec PyCharm** — `odooskills.com/blog/developpement-odoo-2/installer-odoo-20-ubuntu-pycharm-178`.
- **Créer son premier module Odoo 20** — `odooskills.com/blog/developpement-odoo-2/creer-premier-module-odoo-20-179`.

- **Installer Odoo 20 avec Docker** — `odooskills.com/blog/developpement-odoo-2/installer-odoo-20-docker-241`.

7.3 — L'ebook *Développer sur Odoo 20*

Ce guide condense les premiers chapitres de *Développer sur Odoo 20*, qui suit Labo Horizon du premier module au portail patient : relations, héritage, workflows, chatter, rapports, droits avancés, tests, API et composants OWL 3.

> **i Disponible**
>
> L'ouvrage est paru avec la gamme Odoo 20 d'OdooSkills : 523 pages, 21 chapitres et cinq bonus. Il se vend seul ou dans le *Pack Odoo 20*, avec les livres fonctionnel et de déploiement — `odooskills.com/formation-technique-odoo-20`. Un extrait gratuit de 54 pages, avec les chapitres 0 et 1 en entier, permet de juger la méthode avant d'acheter.